

Cross validation

Readings for today

- Chapter 5: Resampling methods. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An introduction to statistical learning: with applications in R (Vol. 6). New York: Springer.

Topics

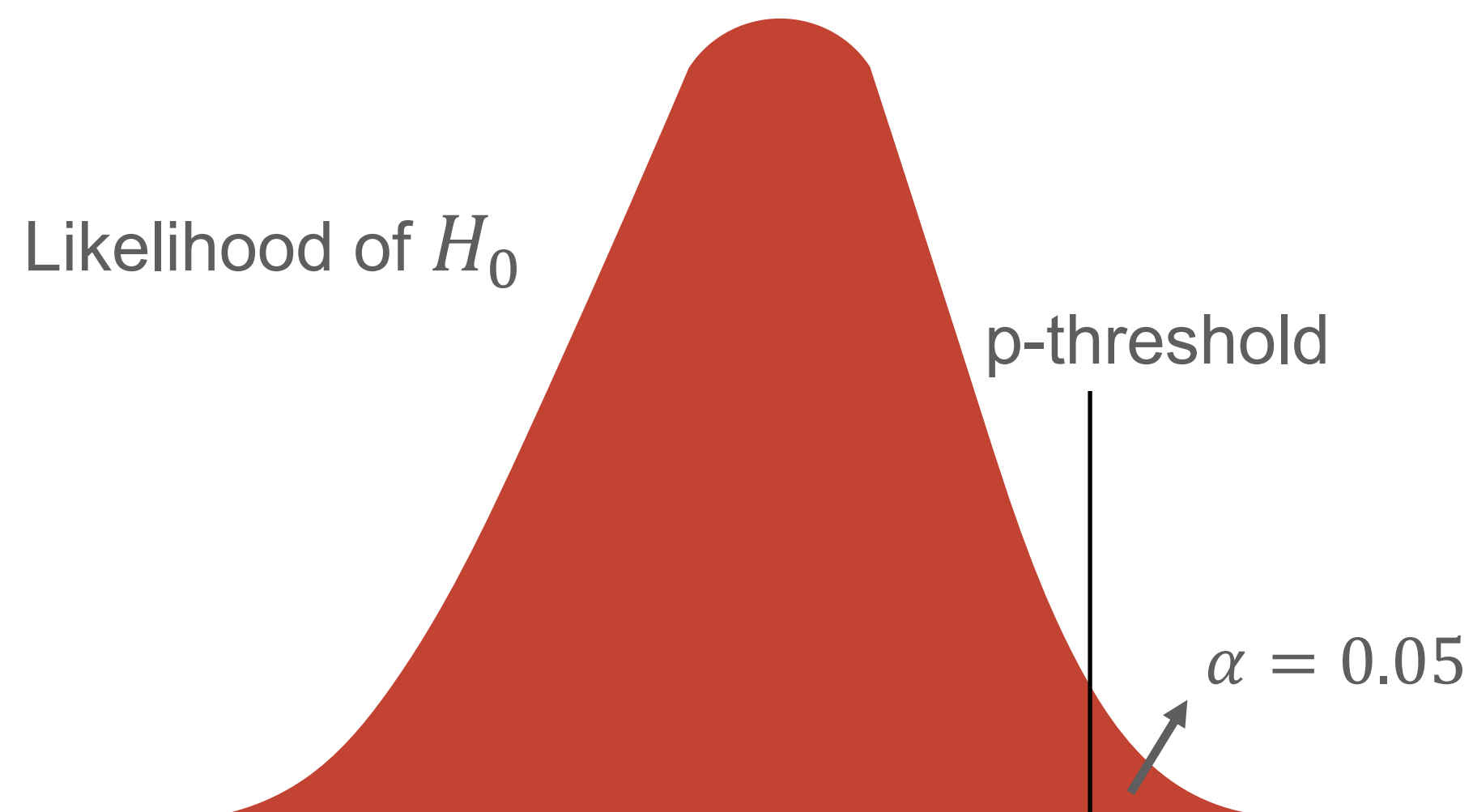
1. Goals of cross validation
2. Leave one out cross validation
3. K-fold cross validation
4. Considerations, use cases, and misuse cases

Goals of cross validation

Inference vs Prediction

Inference

Q: If the null hypothesis ($H_0: \beta = 0$) were true, how surprising would the data we observed be?



$$p = P(|T| \geq |T_{obs}| | H_0 = \text{true})$$

$$p = P(\text{data is at least this extreme} | H_0 = \text{true})$$

Prediction

Q: Does the model ($\hat{y}$) predict observations (y) better than chance (e.g., $\bar{y}$)?

Learn: $\hat{Y} = f_{train}(X_{train})$

Test: $\hat{Y} = f_{train}(X_{test})$

Test Error: $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Compare to baseline model: $\hat{y} = \bar{y}$

Training vs. Test error



Model 1: $\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2$

Model 2: $\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4$

Evaluate: $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Test Error: $\sum_{i=1}^n (y_{test,i} - \hat{y}_{test,i})^2,$
 $\hat{y}_{test,i} = \sum_{j=0}^p \hat{\beta}_j^{train} x_{test,i}$

Training Error: $\sum_{i=1}^n (y_{train,i} - \hat{y}_{train,i})^2,$
 $\hat{y}_{train,i} = \sum_{j=0}^p \hat{\beta}_j^{train} x_{train,i}$

Using prediction to draw inferences

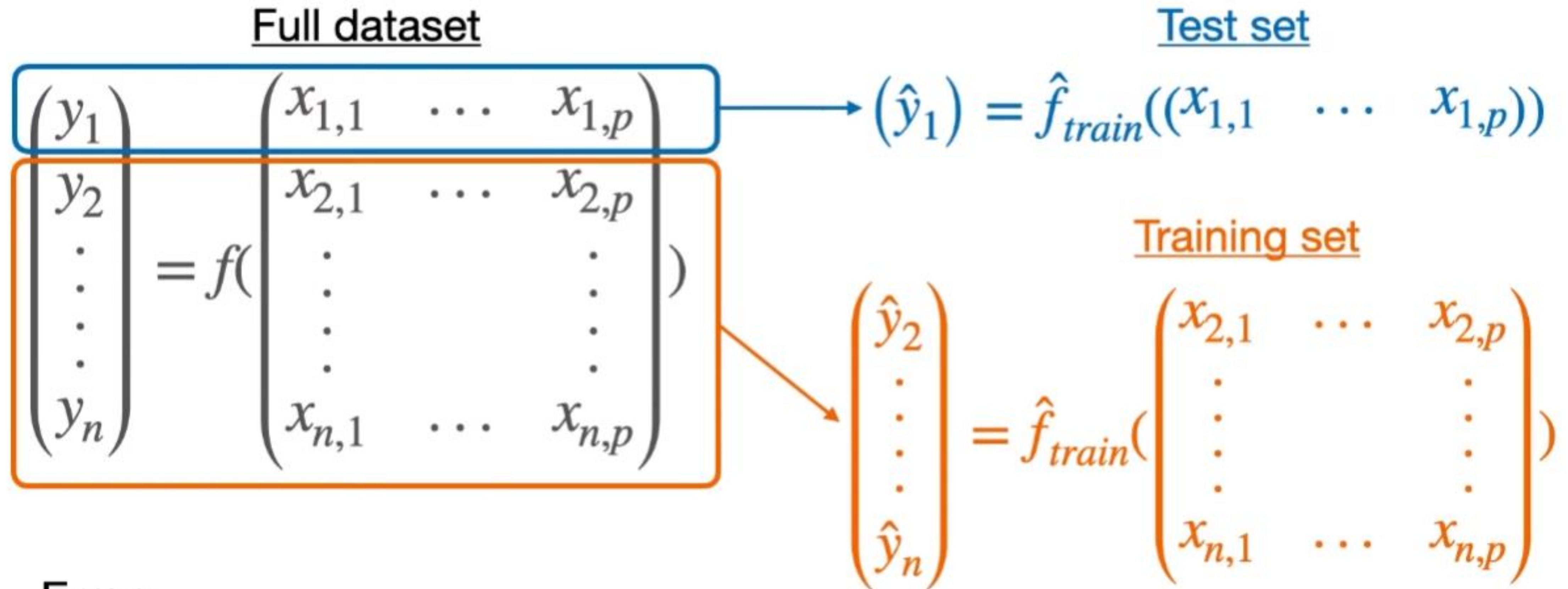


Model 1: $\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$

Model 2: $\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4 + \epsilon$

Can we gain an estimation of generalizability from a single data set?

Simulating generalizability within a dataset for a model



Error:

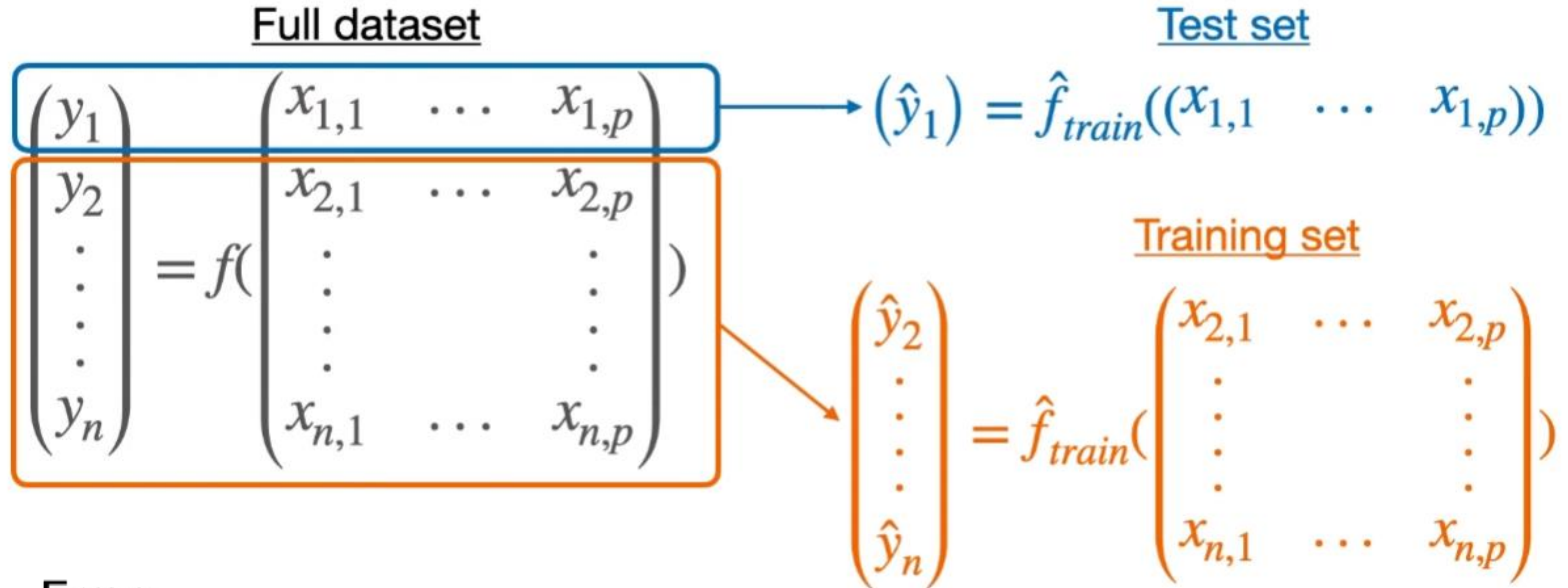
regression: $(y_1 - \hat{y}_1)^2$

classification: $I(y_1 = \hat{y}_1)$

Types of cross-validation approaches

- **Leave one out cross validation (LOOCV)**
- **K-fold cross validation**

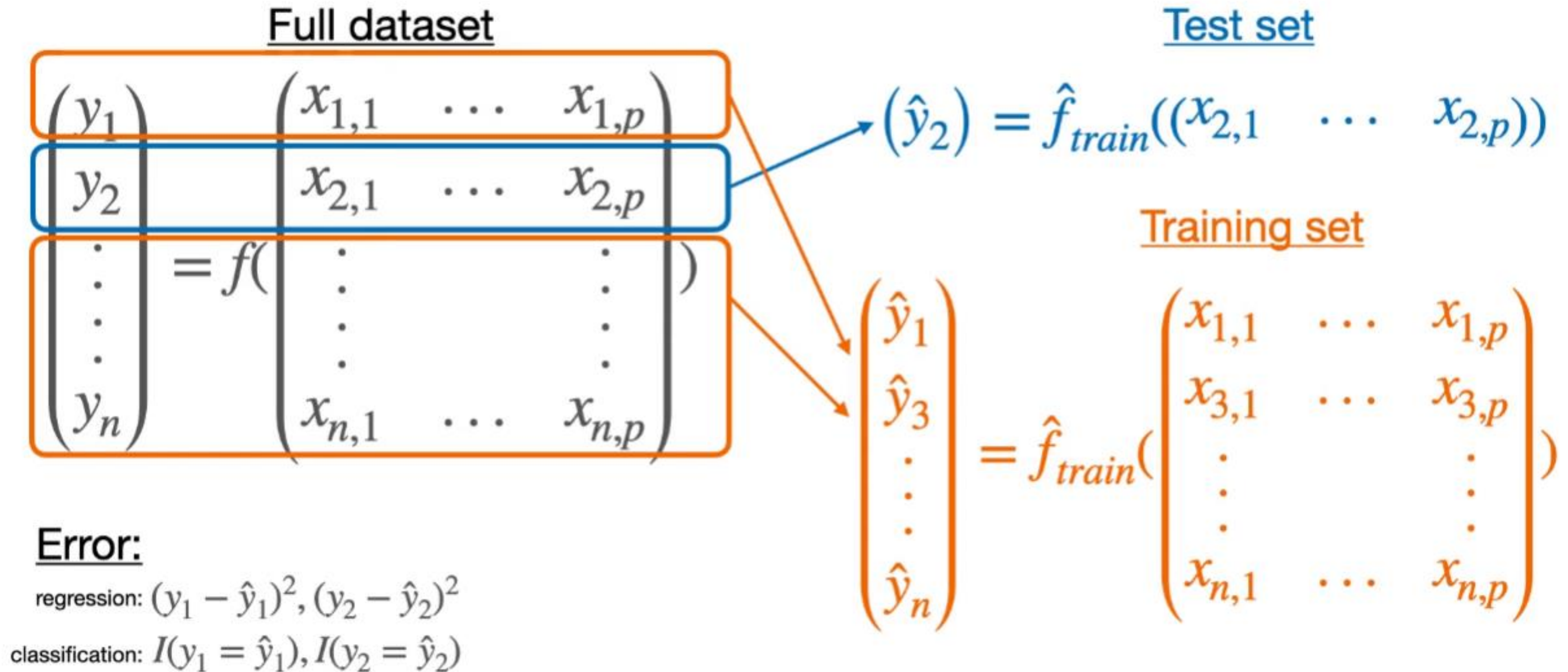
Leave one out cross validation (LOOCV)



Error:

regression: $(y_1 - \hat{y}_1)^2$

classification: $I(y_1 = \hat{y}_1)$



Full dataset

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = f\left(\begin{pmatrix} x_{1,1} & \dots & x_{1,p} \\ x_{2,1} & \dots & x_{2,p} \\ \vdots & & \vdots \\ x_{n,1} & \dots & x_{n,p} \end{pmatrix} \right)$$

Test set

$$(\hat{y}_n) = \hat{f}_{train}((x_{n,1} \quad \dots \quad x_{n,p}))$$

Training set

$$\begin{pmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_{n-1} \end{pmatrix} = \hat{f}_{train}\left(\begin{pmatrix} x_{1,1} & \dots & x_{1,p} \\ \vdots & & \vdots \\ x_{n-1,1} & \dots & x_{n-1,p} \end{pmatrix} \right)$$

Error:

regression: $(y_1 - \hat{y}_1)^2, (y_2 - \hat{y}_2)^2, \dots, (y_n - \hat{y}_n)^2$

classification: $I(y_1 = \hat{y}_1), I(y_2 = \hat{y}_2), \dots, I(y_n = \hat{y}_n)$

LOOCV algorithm

For each candidate model:

Model 1: $\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$
Model 2: $\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4 + \epsilon$

Step1: Take all variables from a single observation,
 $\vec{x}_i = (x_{i,1} \cdots x_{i,p})$ and y_i . Set aside as a test set.

Step2: Make new X_{train} using all observations except i .

Step3: Fit $\hat{f}_{train}(X_{train})$.

Step4: Evaluate test error on y_i using $\hat{f}_{train}$ from Step 3.

Step5: Repeat Steps 1 – 4 for all n observations. Compute *average CV error*.

Select model with lowest *average CV error*

After model selection, refit the chosen model on full data

The good and the bad of LOOCV

Pros:

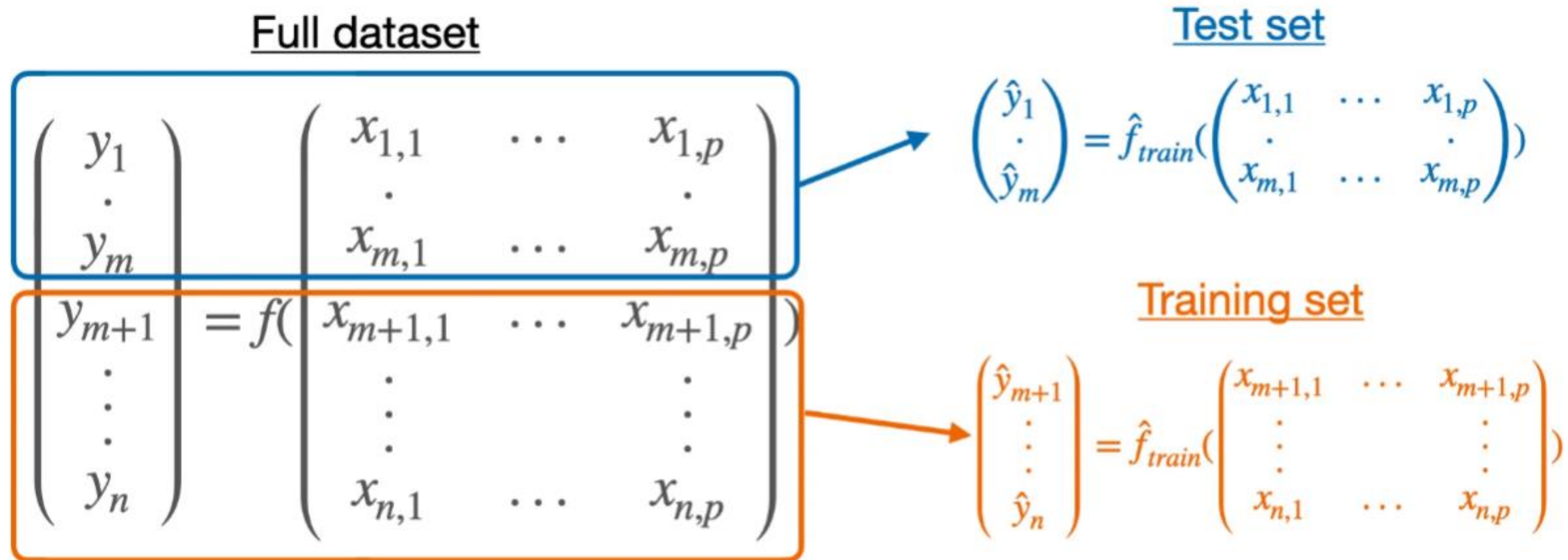
- Uses almost the entire dataset for training
- Stable estimate of $\hat{f}_{train}(X_{train})$
- An estimate of generalization error

Cons:

- Computationally expensive
- Sensitive to high leverage points for small data sets
- Slightly optimistic (negatively biased) estimate of test error in regression

K-fold cross validation

K-fold CV

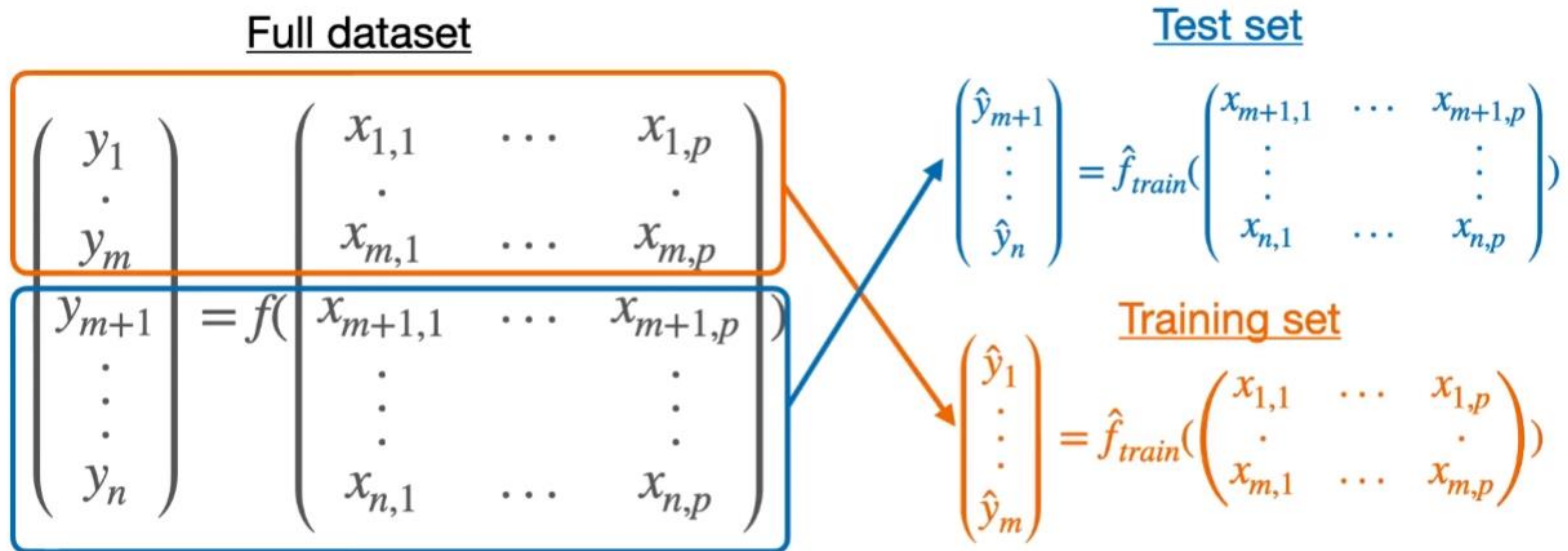


Error:

regression: $(y_1 - \hat{y}_1)^2$

classification: $I(y_1 = \hat{y}_1)$

K-fold CV

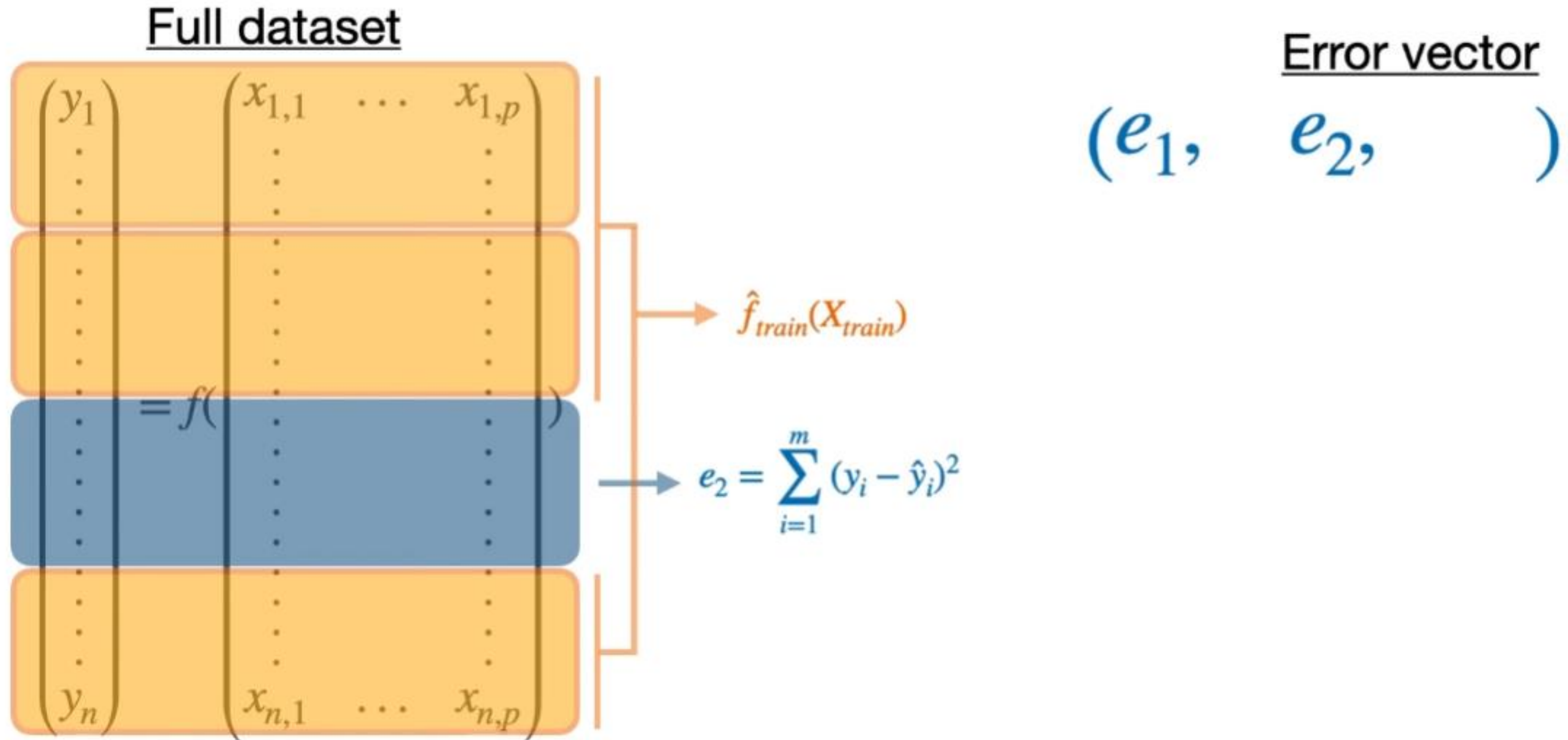


Error:

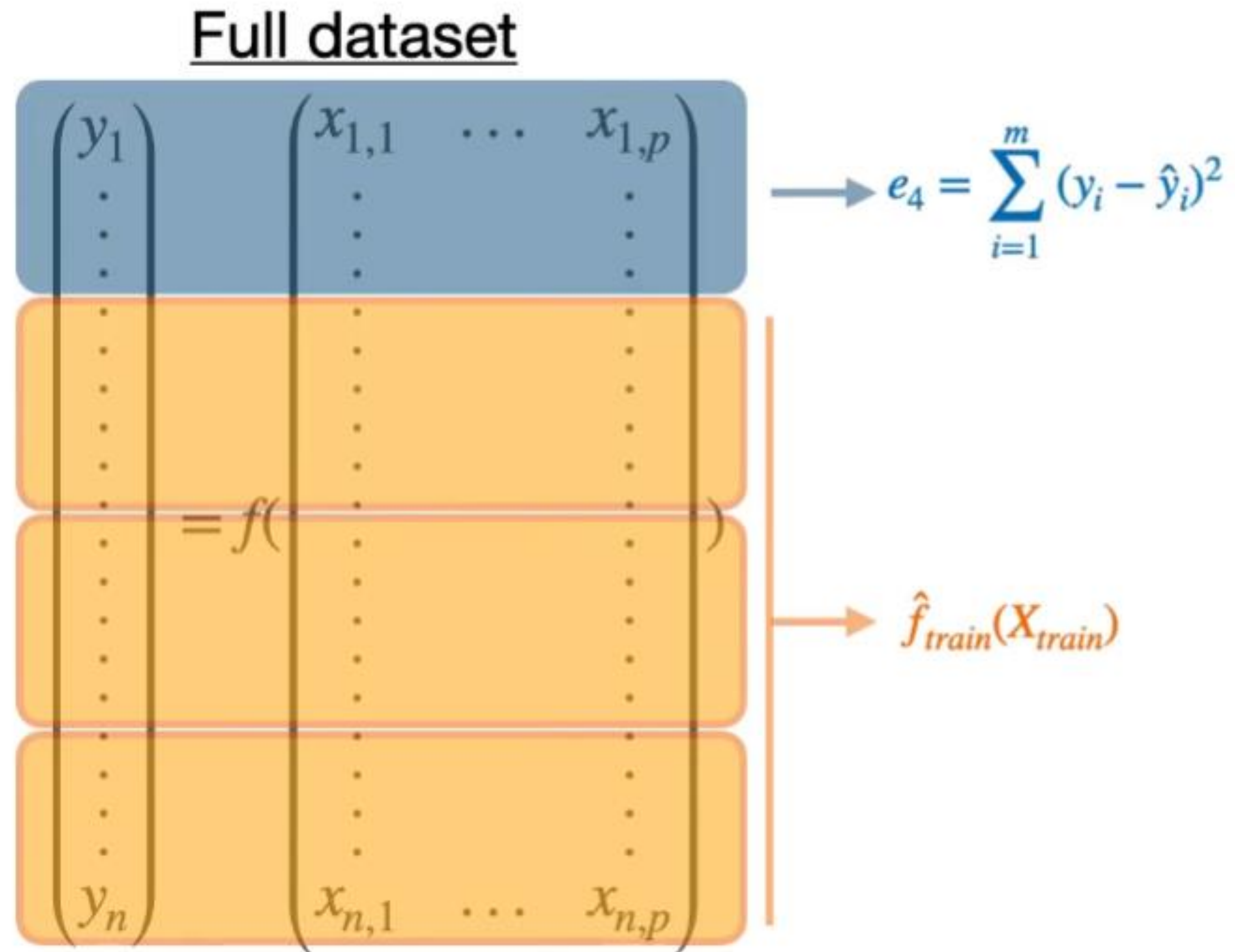
regression: $e_1 = \sum_{i=1}^m (y_i - \hat{y}_i)^2, e_2 = \sum_{i=1}^m (y_i - \hat{y}_i)^2$

classification: $e_1 = \sum_{i=1}^m I(y_i \neq \hat{y}_i), e_2 = \sum_{i=1}^m I(y_i \neq \hat{y}_i)$

4-fold CV example



4-fold CV example



Error vector

(e_1, e_2, e_3, e_4)

CV error:

$$CV_k = \sum_{i=1}^k e_i = \sum_{i=1}^k \sum_{j=1}^m (y_j - \hat{y}_j)^2$$

K-fold CV algorithm

For each candidate model:

Step 1: Take unique subsample of m observations, where $m = \frac{n}{k}$. Set aside as a test set.

Step 2: Make new X_{train} using all observations except m observations.

Step 3: Fit $\hat{f}_{train}(X_{train})$.

Step 4: Evaluate test error on y_i using $\hat{f}_{train}$ from Step 3.

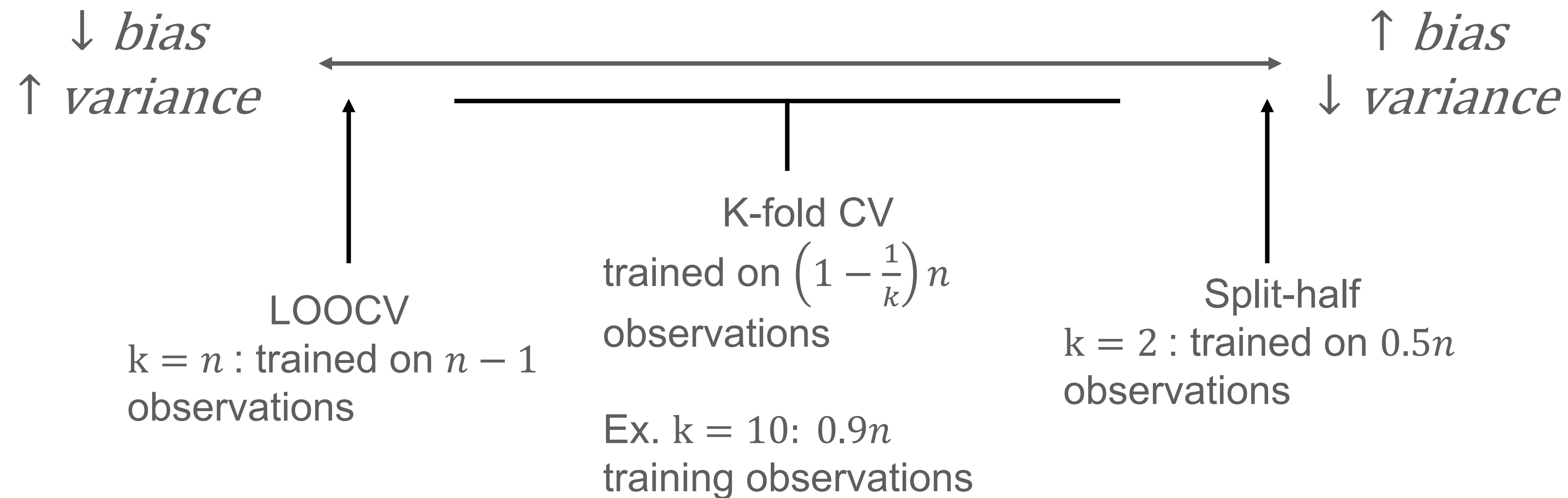
Step 5: Repeat steps 1 - 4 for all k folds.

Step 6: Evaluate full set performance $CV_k = \sum_{i=1}^k \sum_{j=1}^m (y_j - \hat{y}_j)^2$.

Select model with lowest *average CV error*

After model selection, refit the chosen model on full data

Bias-variance tradeoff of cross-validation error estimates



The relationship between k , *bias*, and *variance* is not linear. **5-10 folds** has **slightly more bias** than LOOCV but **reduces variance substantially**

Considerations, use cases, and misuse cases

Things to consider

- Choose k to have enough power in the training set to fit a reliable $\hat{f}(X)$.
- Have enough folds (k) to reduce model variance.
- Balance your subsampling to watch for high-leverage points.
- Watch the dimensionality in the training set.

What CV does (and doesn't do)

What CV does:

- estimates generalization error
- simulates out-of-sample predictions without collecting new data
- compares model strategies (i.e., competing hypotheses)
- helps detect overfitting
- find the best performing hyperparameter

What CV does NOT do:

- fix poor data quality or biased samples
- eliminate overfitting if the modeling process leaks information
- replace a true external test dataset
- guarantee the best model

When should you use CV

- data are limited
- comparing multiple models or alternative hypotheses
- want an estimate of prediction performance
- suspect overfitting
- learn the best hyperparameter, however, to assess stability of hyperparameter **you need to run a nested-CV**

Missuses of traditional CV

- data leakage: performing preprocessing **before splitting folds** (e.g., scale features, dimension reduction, normalize data)
- using CV to estimate inference statistics; CV **does not provide a ‘more reliable’ estimate of effects** (e.g., β)
- making claims about prediction **without a true external test set** (important if you want to publish CV results claiming prediction).
- **tune hyperparameters** (e.g., k , λ for ridge regression, NN learning rate, tree depth for decision tree) **without evaluating hyperparameter stability/uncertainty**. For that, you need to run a nested-CV

Take home message

- Cross validation allows for you to directly test the generalizability of your models without having to collect additional data.

Nested CV: using Inner and Outer folds

Example with $k = 5$ (i.e., 5-folds)

- Step1: Outer fold: Training set (80% of data); Test fold (20%)
- Step2: Inner CV (model tuning): i) run 5-fold CV varying hyperparameter of interest; ii) compare hyperparameter performance; iii) select best hyperparameter
- Step3: Fit the model on the entire outer training set (original 80%)
- Step4: Compute prediction error on the outer test fold.
- Step5: i) Repeat this process for all outer folds. ii) average the outer test errors. iii) that becomes your performance estimate.

You hope to see: **i)** stable outer-fold performance; **ii)** roughly consistent hyperparameters (hyperparameters will remain stable when evaluated against test set); **iii)** similar inner/outer fold performance (implies model is not overfitting)